

Phase 1 Completion Report & Phase 2 Plan

Secure Real-Time Voice Communication System using STM32 and LTE

Phase 1: Analog Talkback Verification with Modified MAX9814 + Headset Microphone **STATUS:**
COMPLETED

Phase 2 (next): STM32 ADC Voice Detection with Onboard LED Indicator

PHASE 1 — Analog Talkback Verification (COMPLETED)

Goal

Verify that all analog audio components — microphone, amplifier, jacks, and headphones — work end-to-end as a continuous signal chain, before introducing any microcontroller. If a person can speak into the microphone and hear themselves in the headphones, all analog hardware is proven good and the project can move on to digital processing.

Approach

Build a pure analog circuit (no STM32 firmware needed) in which the microphone signal is amplified by the MAX9814, AC-coupled through a 1 μ F capacitor into the LM386 power amplifier, and routed through a TRRS jack to standard headphones. The Black Pill is used as a power source only — its 3.3 V and 5 V rails feed the boards.

Components Used

#	Component	Role
1	STM32F401 Black Pill	Power source (3.3 V and 5 V rails). No firmware required.
2	MAX9814 (modified)	Microphone preamplifier with AGC. Onboard electret mic was physically removed.
3	1 μ F capacitor	AC coupling between MAX9814 OUT and LM386 IN to block DC bias.
4	LM386 module	Power amplifier. Drives the headphones from the MAX9814 audio signal.
5	TRRS jack breakout	Female 3.5 mm socket. Provides access to all 4 conductors of the headset plug.
6	TRRS headset	Provides BOTH the microphone (from the headset cable) and the speakers (earbuds).
7	Breadboard + jumpers	Mechanical connection of the above components.

Critical Modification — Onboard Mic Removal

The MAX9814 breakout ships with a small electret microphone soldered to its PCB. On a breadboard, this onboard mic picks up significant environmental noise (50/60 Hz hum, breadboard noise, RF). To improve signal quality, the onboard mic was physically removed and replaced with the higher-quality microphone built into the TRRS headset.

Removal procedure:

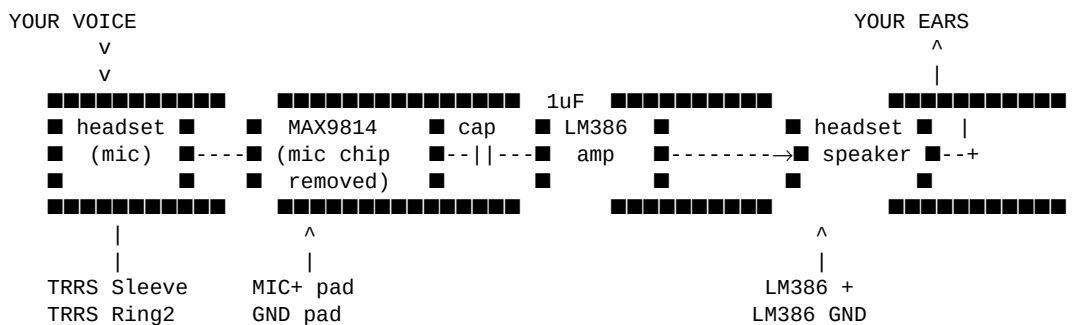
- Located the small cylindrical electret microphone on the MAX9814 board.
- Cut the two leads connecting the mic to the PCB (or desoldered for non-destructive removal).
- Two empty solder pads remained on the PCB where the mic used to sit.
- Identified the two pads using PCB trace inspection AND multimeter continuity:
 - **MIC+** pad — the empty pad whose trace runs toward the MAX9814 chip body. This is the analog input to the chip's internal amplifier.
 - **GND** pad — the empty pad whose trace joins the wider ground copper. Used as the return path.

Headset Microphone Wiring (CTIA standard)

A standard Android/iPhone TRRS headset uses the CTIA pinout. The microphone signal is on the **Sleeve** conductor of the plug, with the microphone's ground return on **Ring 2**. The TRRS jack breakout exposes these as labeled pads:

Plug Section	Carries (CTIA)	Wired to
Tip	Left audio (output)	LM386 + (speaker output, both ears get same signal)
Ring 1	Right audio (output)	LM386 + (same wire as Tip)
Ring 2	Ground	Common GND rail (also serves as headphone return)
Sleeve	Microphone (input)	MAX9814 MIC+ pad (the empty pad after mic removal)

Final Circuit — Block Diagram



Final Circuit — Complete Wire List (14 wires + 1 capacitor)

Section	#	From	To
Power rails	1	Black Pill 3V3 pin	Top + rail of breadboard
	2	Black Pill 5V pin	Bottom + rail of breadboard
	3	Black Pill GND pin	Top – rail of breadboard
	4	Top – rail	Bottom – rail (so all GND is one)
MAX9814 power	5	MAX9814 VDD	Top + rail (3.3 V)
	6	MAX9814 GND	– rail (GND)
Headset mic in	7	TRRS Sleeve pad	MAX9814 MIC+ empty pad
	8	TRRS Ring 2 pad	– rail (GND)
Audio coupling	9	MAX9814 OUT pin	1 μ F cap $\rightarrow$ LM386 IN (header pin)
LM386 power	10	LM386 VCC (header)	Bottom + rail (5 V)
	11	LM386 GND (header)	– rail (GND)
Speaker output	12	LM386 green terminal +	TRRS Tip pad
	13	LM386 green terminal +	TRRS Ring 1 pad
	14	LM386 green terminal –	– rail (GND)

Test Procedure

- Plug Black Pill into PC via USB-C. Red power LED lights up.
- Multimeter check: 3.30 V between top + rail and GND, 4.7–5.1 V between bottom + rail and GND.
- Plug TRRS headset into the TRRS jack breakout.
- Turn LM386 gain potentiometer fully counter-clockwise (minimum gain).
- Hold the headset in the hand — DO NOT put it on the head (mic and earbuds too close → acoustic feedback).
- Hold the headset's mic near the mouth, earbuds at arm's length away from the face.
- Place one earbud loosely near the ear.
- Slowly rotate the LM386 gain knob clockwise while speaking softly.
- When voice becomes audible in the earbud, gain is correctly set.

Result

- ✓ Clear, clean voice reproduction in the earbuds.
- ✓ Significantly lower noise floor compared to the un-modified MAX9814 (with onboard mic).
- ✓ All analog components verified working end-to-end.
- ✓ TRRS pinout confirmed as CTIA standard for the available headset.

Lessons Learned

- **Modular testing pays off.** Verifying the entire analog chain before introducing the microcontroller eliminated several debugging variables for later phases.
- **Onboard mics on breadboards are noisy.** The MAX9814's onboard electret was the dominant noise source; replacing it with the headset mic (which sits at the mouth, far from breadboard noise) dramatically improved SNR.
- **The MAX9814's bias circuit is universal.** Once the onboard mic is removed, the breakout can power any electret microphone connected to its empty pads — no external bias resistor required.
- **Acoustic feedback is unavoidable** when input mic and output speakers share the same headset. The workaround is to hold them physically apart during testing; in the final project the two devices are physically separated.
- **The TRRS jack is purely a connector** — no electronics inside. The 4 pads on the breakout are direct passthroughs to the 4 conductors on the plug (Tip, Ring 1, Ring 2, Sleeve).

PHASE 2 — STM32 ADC Voice Detection (NEXT)

Goal

Prove that the STM32F401 microcontroller can read audio samples from the microphone via its built-in ADC. Provide visual confirmation by lighting the onboard PC13 LED whenever sound is detected by the mic. This bridges the project from pure analog to digital firmware and lays the foundation for all subsequent phases (DAC playback, encryption, LTE transmission).

Why this approach (Option A — LED indicator)

Three options were considered for visualizing the ADC output:

Option	Method	Extra Hardware	Pros	Cons
A (chosen)	Onboard PC13 LED reacts to audio	None	Simplest, no extra parts, immediate visual feedback	No numeric data, only ON/OFF
B	USB-TTL adapter to PC terminal	USB-TTL adapter (CH340 /CP2102)	See actual sample values	Requires extra ~15 LE adapter
C	USB CDC virtual COM via Black Pill USB-C	None	No extra parts	Significantly more CubeMX setup, USB middleware required

Option A is selected for Phase 2 because it gives immediate, unambiguous confirmation that the ADC is functioning, with zero additional hardware. Once Phase 2 passes, Phase 2.5 or Phase 3 can upgrade to Option B or C for full visualization.

Wiring Changes from Phase 1

The audio signal currently flowing into the LM386 needs to be redirected into the STM32's ADC pin (PA0). The LM386 / output speaker portion of the circuit can be physically left in place but is not used in Phase 2.

```
PHASE 1 (was):
    MAX9814 OUT --[1uF cap]--> LM386 IN --> headphones
```

```

PHASE 2 (new):
    MAX9814 OUT -----> PA0 (STM32 ADC input)
                        |
                        v
                    process samples
                        |
                        v
    if (loud) -> turn ON PC13 LED
    if (quiet) -> turn OFF PC13 LED

```

Wires to change

Action	Wire	Notes
REMOVE	1 μ F cap between MAX9814 OUT and LM386 IN	No longer needed; STM32 ADC accepts the DC-biased signal directly.
REMOVE (optional)	LM386 power and output wires	Can leave in place — they will be re-used in later phases.
ADD	MAX9814 OUT $\rightarrow$ STM32 PA0	Direct wire. PA0 is ADC1 channel 0.
KEEP	MAX9814 power (VDD/GND), TRRS Sleeve $\rightarrow$ MAX9814 MIC+ pad, TRRS Ring2 $\rightarrow$ GND	These confirm the headset mic is feeding the MAX9814 just like Phase 1.

CubeMX Configuration

Use the existing SecureVoice CubeMX project (from project documentation), or create a fresh minimal project with only the peripherals needed for Phase 2. The minimum set is:

Peripheral	Setting	Value
RCC	High Speed Clock (HSE)	Crystal/Ceramic Resonator
SYS	Debug	Serial Wire (critical — prevents bricking after first flash)
Clock	HCLK	84 MHz (PLL: M=25, N=336, P=4)
ADC1	IN0 channel	Enabled (PA0)
ADC1	Resolution	8 bits
ADC1	Continuous Conversion	Disabled
ADC1	DMA Continuous Requests	Enabled
ADC1	External Trigger Source	Timer 2 Trigger Out event, Rising edge
ADC1	Sampling Time	15 cycles
ADC1 DMA	Mode	Circular, Peripheral→Memory, Byte/Byte, Memory increment ON
TIM2	Prescaler	83
TIM2	Counter Period	124
TIM2	Trigger Event Selection TRGO	Update Event
GPIO PC13	Mode	Output Push-Pull, Low speed, no pull-up/down, label "LED"
NVIC	Enabled	DMA2 Stream0 IRQ, ADC1 IRQ

8 kHz sampling math: $\text{TIM2 clock} = 84 \text{ MHz} / ((83+1) \times (124+1)) = 8000 \text{ Hz exactly.}$

Firmware (paste into main.c)

Place each block in the correct USER CODE region. The CubeMX-generated init code (HAL_Init, SystemClock_Config, MX_GPIO_Init, MX_DMA_Init, MX_ADC1_Init, MX_TIM2_Init) stays untouched.

Variables and buffers (USER CODE BEGIN PV)

```
/* USER CODE BEGIN PD */
#define BUF_SIZE      256          /* total ping-pong buffer (samples) */
#define HALF_SIZE     (BUF_SIZE / 2)
#define VOICE_THRESHOLD 20        /* peak-to-peak threshold for "voice detected" */
/* USER CODE END PD */

/* USER CODE BEGIN PV */
uint8_t  adc_buffer[BUF_SIZE];
volatile uint8_t half_ready = 0;
volatile uint8_t full_ready = 0;
/* USER CODE END PV */
```

Peripheral start (USER CODE BEGIN 2, inside main())

```
/* USER CODE BEGIN 2 */
HAL_ADC_Start_DMA(&hadc1, (uint32_t*)adc_buffer, BUF_SIZE);
HAL_TIM_Base_Start(&htim2);
/* USER CODE END 2 */
```

Main loop (USER CODE BEGIN WHILE)

```
/* USER CODE BEGIN WHILE */
```

```
while (1)
{
    if (half_ready || full_ready) {
        uint8_t *chunk = half_ready ? &adc_buffer[0]
                                     : &adc_buffer[HALF_SIZE];

        half_ready = 0;
        full_ready = 0;

        /* Compute peak-to-peak amplitude of this chunk */
        uint8_t lo = 255, hi = 0;
        for (int i = 0; i < HALF_SIZE; i++) {
            if (chunk[i] < lo) lo = chunk[i];
            if (chunk[i] > hi) hi = chunk[i];
        }
        uint8_t amp = hi - lo;

        /* Drive PC13 LED – active LOW on the Black Pill */
        if (amp > VOICE_THRESHOLD) {
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET); /* LED ON */
        } else {
            HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_SET);    /* LED OFF */
        }
    }
}
/* USER CODE END WHILE */
```

ADC callbacks (USER CODE BEGIN 4)

```
/* USER CODE BEGIN 4 */
void HAL_ADC_ConvHalfCpltCallback(ADC_HandleTypeDef *hadc) {
    if (hadc->Instance == ADC1) half_ready = 1;
}

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *hadc) {
    if (hadc->Instance == ADC1) full_ready = 1;
}
/* USER CODE END 4 */
```

Test Procedure

- Modify the wiring as described in the 'Wires to change' table above.
- Open the CubeMX project, verify ADC1 + TIM2 + DMA + GPIO PC13 are configured. Generate code.
- Open the project in VS Code (STM32 extension) or STM32CubeIDE.
- Paste the firmware blocks into main.c at the indicated USER CODE regions.
- Build (Ctrl+Shift+B). Confirm 0 errors in the terminal/console.
- Connect ST-Link V2 to the Black Pill (SWDIO ↔ DIO, SWCLK ↔ CLK, GND ↔ GND).
- Power the board via USB-C (do NOT also power from the ST-Link's 3.3 V pin).
- Plug the headset into the TRRS jack — same as Phase 1.
- Click Run/Flash in the IDE. Wait for 'Download verified successfully'.

Expected Results

Stimulus	Expected LED behavior
Silence (no sound)	PC13 LED is OFF
Speaking normally into the headset mic	PC13 LED is ON while speaking, OFF in pauses
Whistling continuously	PC13 LED stays ON
Single sharp clap	PC13 LED briefly flashes
Tapping the headset mic with finger	PC13 LED flickers with each tap

Pass criterion: **the LED reliably reflects voice activity**. The exact threshold value (currently 20) can be tuned: lower for higher sensitivity, higher to reject background noise. If the LED is always on or always off, adjust `VOICE_THRESHOLD` in the firmware and re-flash.

Troubleshooting

Symptom	Likely Cause	Fix
LED always ON, never reacts	ADC reads constant noise above threshold; or ADC stuck	Increase <code>VOICE_THRESHOLD</code> ; verify TIM2 is started; check ADC DMA NVIC enabled
LED always OFF, never reacts	ADC not converting; or PA0 not connected to MAX9814 OUT	Verify PA0 wire; verify ADC1 IN0 is enabled; verify <code>HAL_ADC_Start_DMA</code> was called
LED responds but only to very loud sound	Threshold too high	Lower <code>VOICE_THRESHOLD</code> (try 10)
LED responds to anything, even silence	Threshold too low; or noisy ground	Raise <code>VOICE_THRESHOLD</code> ; verify common ground
Build error: <code>HAL_ADC_Start_DMA</code> undefined	CubeMX did not generate ADC code	Re-open .ioc, ensure ADC1 is enabled, regenerate
Compile OK but flash fails: target locked	SWD was disabled in CubeMX	Set <code>SYS</code> → Debug = Serial Wire, regenerate. If already flashed: hold BOOT0 + reset, mass-erase via STM32CubeProgrammer

What This Phase Proves

- The TIM2 timer is generating a precise 8 kHz trigger.

- The ADC is sampling PA0 at exactly 8 kHz, triggered by TIM2.
- DMA is moving samples from the ADC into RAM in circular mode without CPU intervention.
- Half-complete and full-complete callbacks are firing correctly.
- Main-loop processing of audio frames is running fast enough to handle the data rate.
- GPIO output (PC13 LED) responds correctly to firmware control.

These are exactly the same primitives that will be used in every later phase, just with different actions in the main loop (encryption, transmission, etc.). Once Phase 2 passes, the digital input pipeline is verified.

After Phase 2

Once the LED reliably reacts to voice, the next milestones are:

- **Phase 2.5** — Upgrade to PC visualization (Option B or C) so actual sample values can be inspected.
- **Phase 3** — Add the MCP4725 DAC: ADC samples are written to the DAC at 8 kHz, producing a digital-domain mic-to-speaker loopback (full audio path running through the STM32).
- **Phase 4** — LTE module bring-up with manual AT commands.
- **Phase 5** — STM32 drives the LTE module, connects to the relay server, performs phone-number-based call setup.
- **Phase 6** — Plain audio over LTE between two devices (no encryption).
- **Phase 7** — Add RSA + AES hybrid encryption to the audio stream.
- **Phase 8** — Polish: physical buttons for call/answer, LED status states, demo prep.

End of Phase 1 / Phase 2 document.